```
print preg_filter($patterns, $replacements, $text);
```

```
Output:
The slow tortoise overtook the quick hare.
```

This is where `preg_filter()` shows that it has a lot more to offer than the simple byte comparison and swapping of `str_replace()`. The `$patterns` array contains expressions that are applied in the order they are defined in the array (not key order!), and the corresponding array elements of `$replacements` are used upon a successful match. The replacement strings are also expressions. In this case we're using the `${n}` form, which is preferred over the `\n` syntax; be aware that named and relative capture groups are not supported. The sequential application of the matching and substitution means we can pull tricks like this:

```
$text = 'The goldfish overtook the shark.';
$patterns = [
    '/(goldfish)/',
    '/(nimble)/'
];
$replacements = [
    'nimble ${1}',
    '${1}, quick',
];

print preg_filter($patterns, $replacements, $text);
```

```
Output:
The nimble, quick goldfish overtook the shark.
```

The first pass applies the `/(goldfish)/` pattern to the sample. There's a match, so the sample immediately becomes `'The nimble goldfish overtook the shark.'`. The second pass uses the expression `/(nimble)/`, which the sample string now contains, so the second replacement is triggered, as we see in the final output. If there are more elements in `$patterns` than in `$replacements`, then those patterns that match in the sample but don't have a corresponding replacement will instead be replaced by an empty string. Here's what happens when we remove the second replacement from the example above: